Android - Unit 4

UI Architecture

User Interfaces 2

Unit 4: UI architecture

1. Architecture Components: this week
2. Navigation in Jetpack Compose
3. Adapt for different screen sizes

PATHWAY 1



PW1 Architecture Components (link)

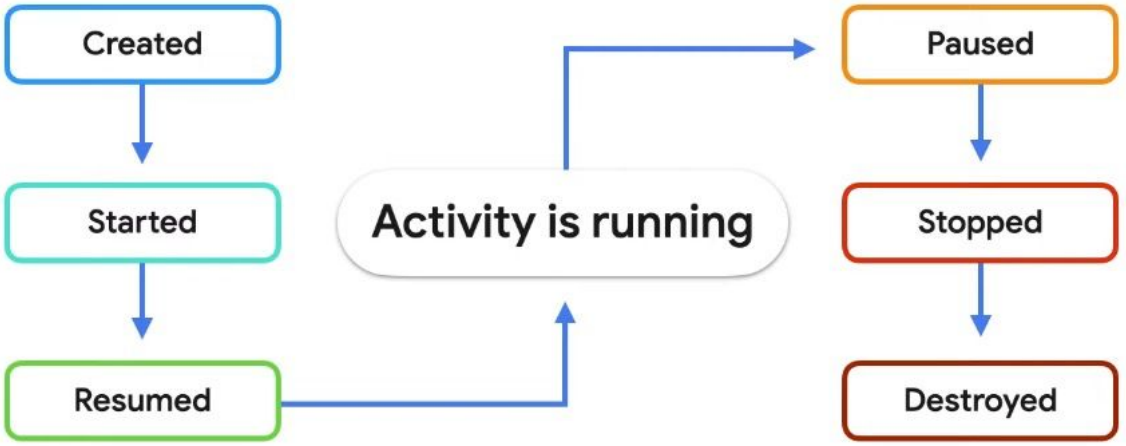


- Stages of the activity lifecycle
- Architecture: The UI Layer
  - ViewModel and State in Compose



(Unit testing)

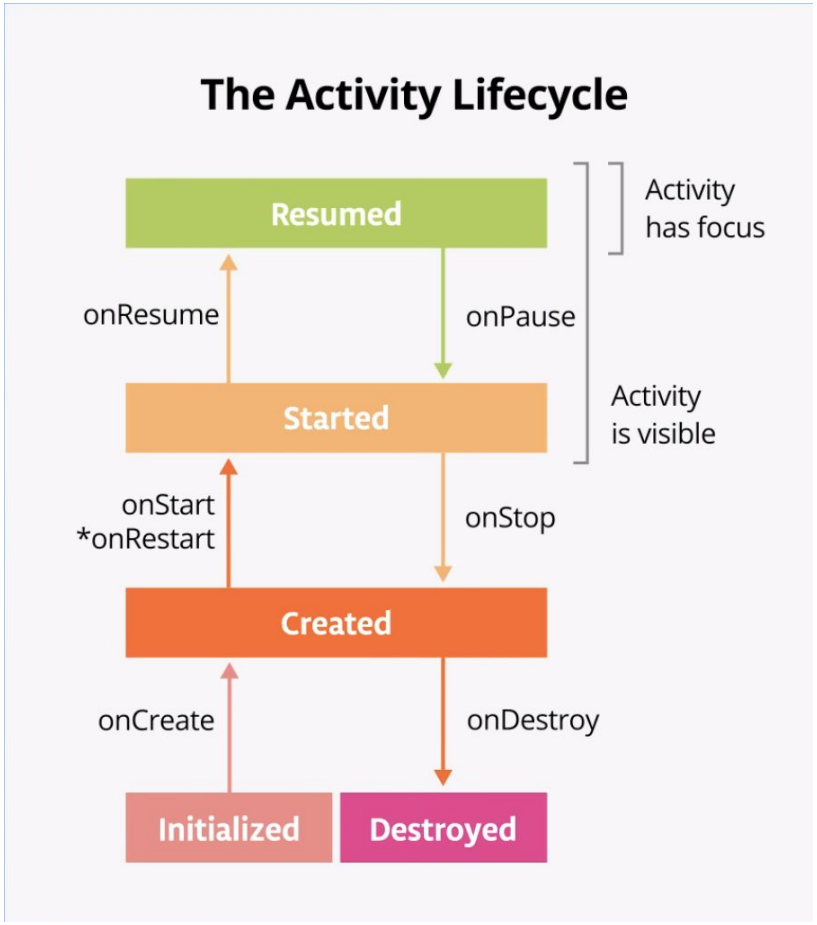
Stages of the activity lifecycle



Stages of the activity lifecycle



- Callback methodes bij verschillende lifecycle stages.
  - `onCreate()`
  - `onStart()`
  - `onResume()`
  - `onPause()`
  - `onStop()`
  - `onDestroy()`



Stages of the activity lifecycle



Logging

- De instructies bevatten vrij repetitieve code, waar je steeds de klasse (in een const TAG) en methode moet meegeven:

```
Log.i(TAG, "onCreate Called")
```

- Je kan (in een klasse) ook klasse en methode dynamisch opvragen en de log context generiek maken :

```
Log.i(javaClass.simpleName,
  "${object{}.javaClass.enclosingMethod?.name} called")
```

- Als derde parameter kan je ook een Throwable meegeven

Stages of the activity lifecycle



Configuration changes

- Als er een verandering in de state van je toestel gebeurd.
  - Orientation change (landscape <> portrait)
  - Device language change
  - Plugin a physical keyboard

- Configuration Change → `onDestroy()` wordt aangeroepen.
  - Data loss ….. ? wat nu ?
    - `rememberSaveable` i.p.v. `remember`

Dit houdt logica in **Composable**. Kan je gebruiken als de data in geen andere Composable gebruikt worden

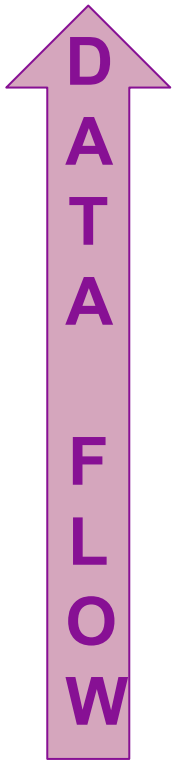
⚠ Let op: Dit geeft geen **separation of concerns** (logica is niet de taak van de UI): voor complexere apps gebruik je beter een gelaagde architectuur (volgende slide)

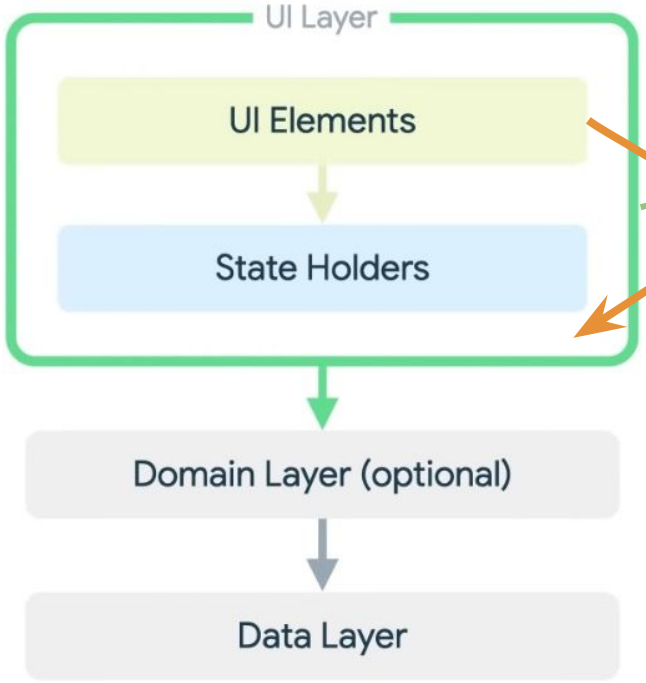
Architecture: The UI Layer



- Unidirectional Data Flow Pattern (UDF)
  - Meer uitleg UDF

De **UI** layer is de visuele representatie van de application state in de **Data Layer**



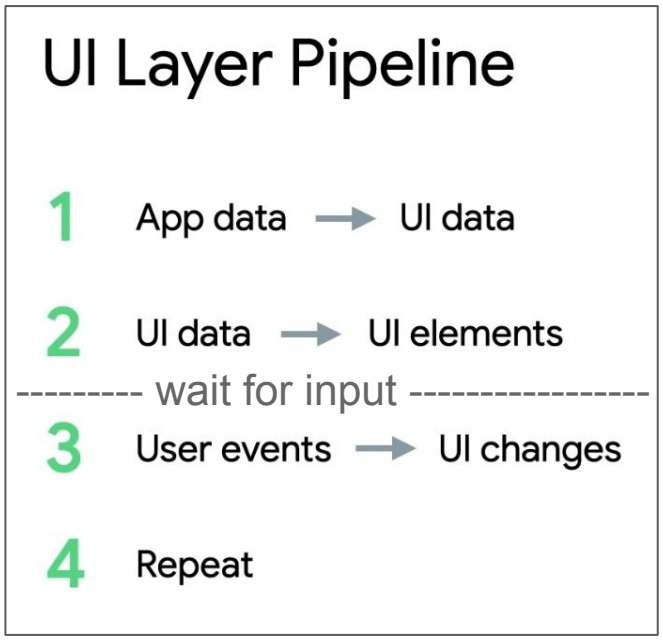


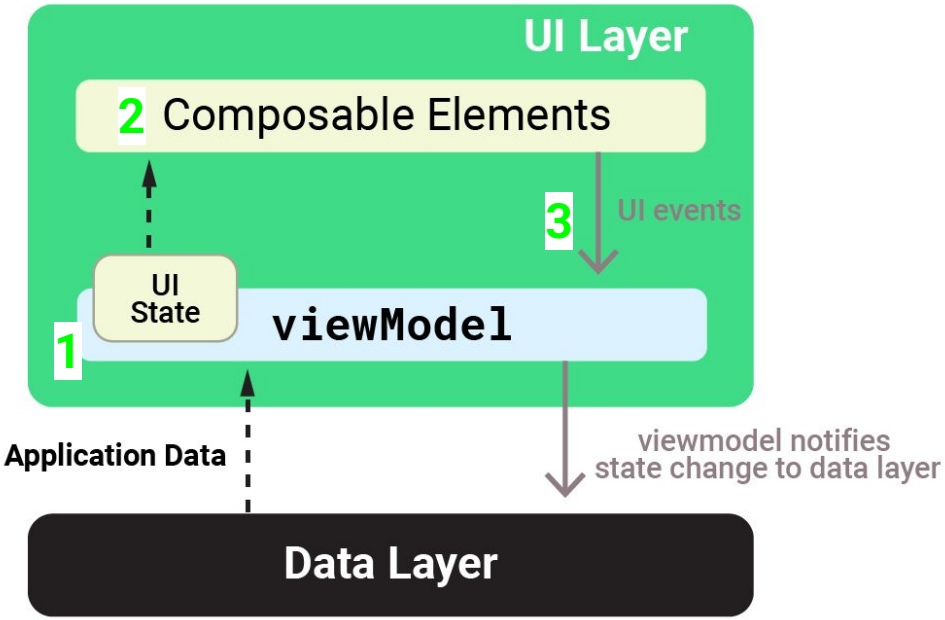


Architecture: The UI Layer



- ViewModel
  - Holds UI State en exposes to Compose UI
    - UI State object is immutable (maak een nieuw object bij veranderingen)
  - UI logica (en optioneel business logica)

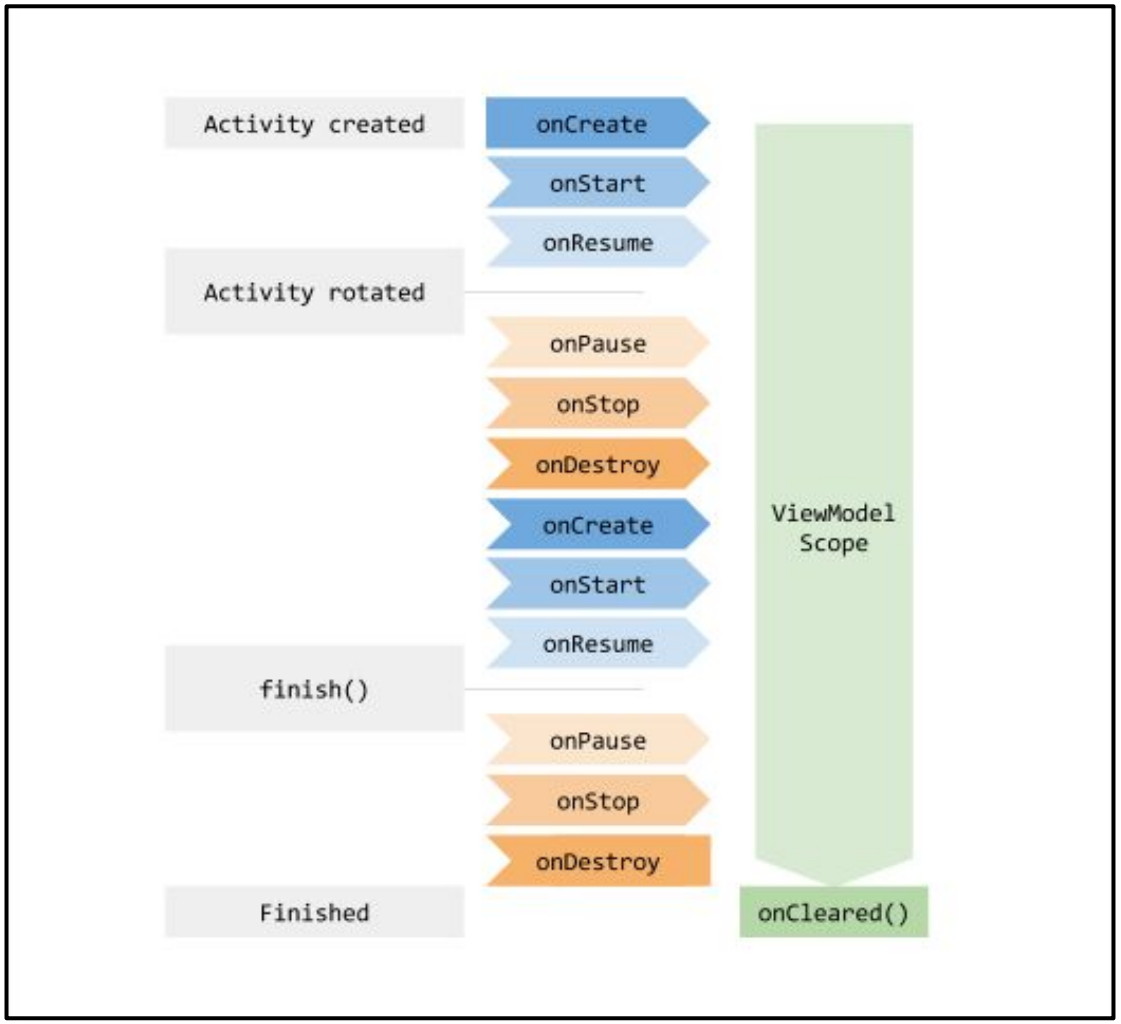




Architecture: The UI Layer



ViewModel heeft een veel langere lifecycle dan Activity



ViewModel and State in Compose



- ViewModel & UIstate data class
  - Data overleeft Configuratie aanpassingen (zoals draaien)

Kotlin StateFlow
  - een **immutable observable** data holder (reactief model)
    - zelfde rol als eerder geziene `MutableState`
    - value property bevat altijd de laatste data in de pipeline
  - Pas centraal aan in een ***`MutableStateFlow`***, die zijn huidige inhoud **`asStateFlow()`** kan uitreiken

- Voeg dependency toe in je **module** gradle bestand (+ Sync)

```
implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.7")
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
```

ViewModel and State in Compose



In ViewModel

1. Gebruikt application data. State zit in een of meer aparte (data) klassen.

private _backingproperty ***MutableStateFlow*** maakt dat ze enkel hier aanpasbaar zijn

```
private val _uiState = MutableStateFlow(JeUiState())
// 2. uiState immutable StateFlow: UIState -> UIElements
val uiState: StateFlow<JeUiState> = _uiState.asStateFlow()
```

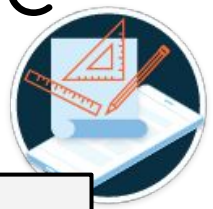
Je State zelf is immutable:

update in ***MutableStateFlow*** door een kopie te maken

de lambda `it` parameter is _uistate.value

```
_uiState.update { it.copy(
        stateAttribute1 = <nieuwe waarde>,
        stateAttribute2 = <nieuwe waarde>,
    )
}
```

ViewModel and State in Compose



Kotlin note

```
// lateinit: variabele die je niet bij declaratie initialiseert
// wel voor gebruik!
private lateinit var currentWord: String
```

ViewModel and State in Compose



In Composable

2. Je ViewModel via constructor *injecteren*
Je registreert je via een delegate (`by`) als Observer van de `uiState`,
zodat een uiState aanpassing recomposition triggert (***UIState -> UIElements***)

Creates or retrieves viewModel from its scope

```
@Composable
fun MyComposable(myViewModel: MyViewModel = viewModel()) {
    val myUIState by myViewModel.uiState.collectAsState()
    //...
}
```

Bij gebruik van `by` delegates steeds getValue importeren

```
import androidx.compose.runtime.getValue
```

3. Events roepen methoden ViewModel aan

```
Button(
 onClick = { viewModel.checkGuess() }
)
```

PW1 Architecture Components (link)





(Activity 6: Write unit tests to test the ViewModel)

Practice: Add a ViewModel to Dessert Clicker

➔ Pas UI architectuur principes toe op de app die eerder gebruikt werd in het Activity Lifecycle codepath



Te behalen Badge Pathway 1:
https://developer.android.com/courses/pathways/android-basics-compose-unit-4-pathway-1